MAD Batch 2

B.E / B.Tech. PRACTICAL END SEMESTER EXAMINATIONS

Sixth Semester

IT3681 - MOBILE APPLICATIONS DEVELOPMENT LABORATORY

(Regulations 2021) - SET 2 / BATCH 2 (SIMPLIFIED)

Time: 3 Hours | **Max. Marks:** 100

MARK ALLOCATION TABLE

AIM	ALGORITHM / UI DESIGN	PROGRAM / CODE	EXECUTION & OUTPUT	VIVA	RECORD	TOTAL
10	20	30	20	10	10	**100**

(Note: Some questions in Batch 2 overlap completely. They have been grouped together below so you know exactly which code to write for your specific question number).

Ex 1: Update ProgressBar from Background Thread

Aim: To write an application that uses a Handler or Thread to update a ProgressBar from a background thread.

Procedure:

1. Add a `ProgressBar` in `activity_main.xml`.
2. In `MainActivity.java`, start a new `Thread`.
3. Use `runOnUiThread` or a `Handler` to update the progress value safely.

Program (Java):

```
ProgressBar progressBar = findViewById(R.id.progressBar);
new Thread(new Runnable() {
    public void run() {
        for (int i = 0; i <= 100; i += 10) {
```

```
            final int progress = i;
            runOnUiThread(new Runnable() {
                public void run() {
                    progressBar.setProgress(progress);
                }
            });
            try { Thread.sleep(500); } catch (InterruptedException e) {
e.printStackTrace(); }
        }
    }
}).start();
```

Output:

```
A progress bar appears on screen and smoothly fills from 0% to 100%.
```

Result: The background thread successfully updated the UI ProgressBar.

Ex 2: Application using Multi-threading

Aim: To implement an Android application that uses multi-threading.
Procedure:

1. Create a UI with two `TextView` elements.
2. Create two separate `Thread` objects in Java.
3. Start both threads simultaneously to update the text views at different speeds.

Program (Java):

```
TextView t1 = findViewById(R.id.text1);
TextView t2 = findViewById(R.id.text2);

new Thread(() -> {
    for(int i=0; i<5; i++) {
        runOnUiThread(() -> t1.setText("Thread 1: " +
System.currentTimeMillis()));
        try { Thread.sleep(1000); } catch(Exception e){}
    }
}).start();

new Thread(() -> {
    for(int i=0; i<5; i++) {
        runOnUiThread(() -> t2.setText("Thread 2: " +
System.currentTimeMillis()));
        try { Thread.sleep(2000); } catch(Exception e){}
```

```
    }
}).start();
```

Output:

```
Thread 1 updates every 1 second.
Thread 2 updates every 2 seconds independently.
```

Result: Multi-threading was successfully implemented.

Ex 3 & Ex 11: GPS Location Coordinates

(Write this for Question 3 OR Question 11)

Aim: To develop an application that retrieves the user's current GPS coordinates (Latitude and Longitude) and displays them.
Procedure:

1. Add `ACCESS_FINE_LOCATION` permission in `AndroidManifest.xml`.
2. Use `LocationManager` to request location updates.
3. Display `location.getLatitude()` and `location.getLongitude()` in a TextView.

Program (Java):

```
LocationManager lm = (LocationManager)
getSystemService(Context.LOCATION_SERVICE);
LocationListener listener = new LocationListener() {
    public void onLocationChanged(Location loc) {
        TextView tv = findViewById(R.id.tvLocation);
        tv.setText("Lat: " + loc.getLatitude() + " | Lon: " +
loc.getLongitude());
    }
};
// Assuming permissions are already granted
lm.requestLocationUpdates(LocationManager.GPS_PROVIDER, 0, 0, listener);
```

Output:

```
Lat: 13.0827 | Lon: 80.2707
```

Result: The GPS coordinates were successfully retrieved and displayed.

Ex 4: Write Data to SD Card

Aim: To implement an application that writes user input to the SD card (External Storage).
Procedure:

1. Add `WRITE_EXTERNAL_STORAGE` permission in Manifest.
2. Take string input from an `EditText`.
3. Use `FileOutputStream` with `getExternalFilesDir()` to save the text file.

Program (Java):

```
String data = "Sample Data for SD Card";
File file = new File(getExternalFilesDir(null), "mydata.txt");
try {
    FileOutputStream fos = new FileOutputStream(file);
    fos.write(data.getBytes());
    fos.close();
    Toast.makeText(this, "Saved to SD Card!", Toast.LENGTH_SHORT).show();
} catch (Exception e) {
    e.printStackTrace();
}
```

Output:

```
Toast: Saved to SD Card! (File mydata.txt created in Android/data/folder).
```

Result: Data was successfully written to the external SD card.

Ex 5 & Ex 14: Broadcast Receiver for Incoming SMS

(Write this for Question 5 OR Question 14)

Aim: To implement an application that listens for incoming SMS messages and creates an alert dialog.
Procedure:

1. Add `RECEIVE_SMS` permission in `AndroidManifest.xml`.
2. Create a class extending `BroadcastReceiver`.
3. Extract message from `intent.getExtras()` and trigger an `AlertDialog`.

Program (Java):

```
public class SMSReceiver extends BroadcastReceiver {
    public void onReceive(Context context, Intent intent) {
```

```
        Object[] pdus = (Object[]) intent.getExtras().get("pdus");
        SmsMessage msg = SmsMessage.createFromPdu((byte[]) pdus[0]);
        String body = msg.getMessageBody();

        Toast.makeText(context, "New SMS: " + body,
Toast.LENGTH_LONG).show();
        // (If inside an Activity, use AlertDialog.Builder here instead of
Toast)
    }
}
```

Output:

```
Popup Alert: New SMS: Hello, this is a test message!
```

Result: SMS Broadcast Receiver successfully created an alert.

Ex 6: Repeated Notification

Aim: To implement an application that triggers a repeated notification every 60 seconds until manually canceled.

Procedure:

1. Use `AlarmManager` and a `PendingIntent`.
2. Set repeating alarm for 60000ms (60 seconds).
3. In the Receiver, use `NotificationManager` to build and issue the notification.

Program (Java):

```
AlarmManager am = (AlarmManager) getSystemService(Context.ALARM_SERVICE);
Intent intent = new Intent(this, MyReceiver.class);
PendingIntent pi = PendingIntent.getBroadcast(this, 0, intent, 0);

// Repeat every 60 seconds
am.setRepeating(AlarmManager.RTC_WAKEUP, System.currentTimeMillis(),
60000, pi);

// To Cancel:
// am.cancel(pi);
```

Output:

```
A notification drops down in the status bar every 1 minute.
```

Result: Repeated notification executed successfully.

Ex 7 & Ex 12: Parse RSS Feed

(Write this for Question 7 OR Question 12)

Aim: To write a program to fetch and parse an RSS Feed from a news website and display the headlines.

Procedure:

1. Add `INTERNET` permission in Manifest.
2. Run a background thread using `HttpURLConnection` to fetch XML.
3. Use `XmlPullParser` to extract `<title>` tags and display in a `ListView`.

Program (Java):

```
new Thread(() -> {
    try {
        URL url = new URL("
[https://timesofindia.indiatimes.com/rssfeeds/-2128936835.cms]
(https://timesofindia.indiatimes.com/rssfeeds/-2128936835.cms)");
        HttpURLConnection conn = (HttpURLConnection) url.openConnection();
        InputStream in = conn.getInputStream();

        // Simple manual parsing simulation
        BufferedReader br = new BufferedReader(new InputStreamReader(in));
        String line;
        while ((line = br.readLine()) != null) {
            if (line.contains("<title>")) {
                System.out.println("Headline: " + line);
            }
        }
    } catch (Exception e) {}
}).start();
```

Output:

```
Headline: Latest News Updates...
```

Result: RSS feed successfully fetched and parsed.

Ex 8 & Ex 13: Send Email / Email Feedback Form

(Write this for Question 8 OR Question 13)

Aim: To create an app with an "Email Feedback" form that opens the default email client with pre-filled details.

Procedure:

1. Create a UI with `EditText` for Subject and Message.
2. Use `Intent(Intent.ACTION_SENDTO)` with a `mailto:` URI.
3. Put `EXTRA_SUBJECT` and `EXTRA_TEXT` extras.

Program (Java):

```
Intent emailIntent = new Intent(Intent.ACTION_SENDTO);
emailIntent.setData(Uri.parse("mailto:support@college.edu"));
emailIntent.putExtra(Intent.EXTRA_SUBJECT, "App Feedback");
emailIntent.putExtra(Intent.EXTRA_TEXT, "The app is working perfectly.");

startActivity(Intent.createChooser(emailIntent, "Send Feedback"));
```

Output:

```
Gmail app opens automatically with "support@college.edu" in the To field
and the message pre-typed.
```

Result: Email intent successfully launched the email client.

Ex 9: Mobile Application for Simple Needs

Aim: To develop a simple mobile application (e.g., Simple Addition Calculator).

Procedure:

1. Add two `EditText` fields, a `Button`, and a `TextView` in XML.
2. Get numbers on button click, add them, and set text to result.

Program (Java):

```
EditText e1 = findViewById(R.id.num1);
EditText e2 = findViewById(R.id.num2);
Button btn = findViewById(R.id.addBtn);
TextView res = findViewById(R.id.result);

btn.setOnClickListener(v -> {
    int a = Integer.parseInt(e1.getText().toString());
    int b = Integer.parseInt(e2.getText().toString());
```

```
    res.setText("Total: " + (a + b));
});
```

Output:

```
User enters 5 and 10. Clicks Add. Output: "Total: 15"
```

Result: Simple needs application developed successfully.

Ex 10: Alarm Clock Application

Aim: To write a mobile application that creates an alarm clock.
Procedure:

1. Use a `TimePicker` to select the alarm time.
2. Use `AlarmManager` to schedule a pending intent at the chosen time.
3. Play a `Ringtone` inside the BroadcastReceiver.

Program (Java):

```
// Scheduling the alarm
AlarmManager am = (AlarmManager) getSystemService(ALARM_SERVICE);
Intent intent = new Intent(this, AlarmReceiver.class);
PendingIntent pi = PendingIntent.getBroadcast(this, 0, intent, 0);

Calendar c = Calendar.getInstance();
c.set(Calendar.HOUR_OF_DAY, 6); // 6 AM
c.set(Calendar.MINUTE, 0);

am.setExact(AlarmManager.RTC_WAKEUP, c.getTimeInMillis(), pi);

// Inside AlarmReceiver.java:
// Ringtone r = RingtoneManager.getRingtone(context,
RingtoneManager.getDefaultUri(RingtoneManager.TYPE_ALARM));
// r.play();
```

Output:

```
Alarm scheduled for 6:00 AM. Ringtone plays at exact time.
```

Result: Alarm clock successfully created.

Ex 15: Weather Info Mini-Project

Aim: To combine GUI, Networking, and Notifications to fetch weather data and alert the user.
Procedure:

1. Add `INTERNET` permission.
2. Create GUI with "Get Weather" button.
3. Use a Background thread to fetch JSON from OpenWeather API.
4. If the JSON condition says "Rain", trigger a `NotificationManager` alert.

Program (Java Overview):

```
new Thread(() -> {
    // 1. Networking (Fetch JSON)
    String result = fetchFromURL("
[http://api.weatherapi.com/v1/current.json?key=XYZ&q=Chennai]
(http://api.weatherapi.com/v1/current.json?key=XYZ&q=Chennai)");

    // 2. GUI Update
    runOnUiThread(() -> weatherTextView.setText(result));

    // 3. Notification
    if (result.contains("Rain")) {
        NotificationCompat.Builder builder = new
NotificationCompat.Builder(this, "CH1")
            .setSmallIcon(R.drawable.ic_rain)
            .setContentTitle("Weather Alert")
            .setContentText("It's raining! Take an umbrella.");
        NotificationManager nm = (NotificationManager)
getSystemService(NOTIFICATION_SERVICE);
        nm.notify(1, builder.build());
    }
}).start();
```

Output:

```
GUI shows "Temp: 28C, Condition: Rain".
Push Notification appears saying "It's raining! Take an umbrella."
```

Result: Weather mini-project executed successfully.

Ex 16: Contact Manager Mini-Project

Aim: To use a Database to store names, GPS to tag locations, and an option to Email details.

Procedure:

1. Create `SQLiteOpenHelper` to save Name, Phone, Lat, Lon.
2. Use `LocationManager` to get coordinates when clicking "Save Contact".
3. Add an "Email" button that queries the SQLite database and uses `ACTION_SEND` Intent.

Program (Java Overview):

```
// Save Contact with GPS
db.execSQL("INSERT INTO Contacts VALUES ('John', '98765', " + lat + ", " +
lon + ")");

// Email Button Logic
Cursor c = db.rawQuery("SELECT * FROM Contacts WHERE Name='John'", null);
if(c.moveToFirst()) {
    String details = "Name: " + c.getString(0) + "\nPhone: " +
c.getString(1) +
                     "\nLocation: " + c.getString(2) + ", " +
c.getString(3);

    Intent emailIntent = new Intent(Intent.ACTION_SEND);
    emailIntent.setType("text/plain");
    emailIntent.putExtra(Intent.EXTRA_TEXT, details);
    startActivity(emailIntent);
}
```

Output:

```
Contact saved with GPS.
Clicking 'Email' opens Gmail with contact details and location auto-filled
in the body.
```

Result: Contact Manager mini-project executed successfully.
